GPU-Accelerated Computational Lithography Simulation

Team Members: Yash Agarwal, Daniel Fields

Project URL: https://yash-agarwal.org/418/

Summary

We plan to implement and optimize a GPU-based computational lithography simulator using C++ and CUDA. We want to first implement a baseline simulator using full-resolution FFTs, and then implement a more efficient algorithm that reduces the sizes of the intermediate FFTs. We plan to experiment with several parallelization strategies, including batching optical computations, parallel reductions, memory layout optimizations, and kernel fusion, and compare their performance on NVIDIA GPUs.

Background

Computational lithography is used in semiconductor manufacturing to predict how a pattern on a photomask will be printed onto a silicon wafer. Because of diffraction and other optical effects, the printed pattern will not exactly match the mask, and lithography simulation is used to predict these differences. This simulation is an important part of optical proximity correction (OPC) and inverse lithography technology (ILT), which repeatedly simulate and modify a mask to produce a pattern that more closely matches the target.

We will focus on implementing the forward lithography simulation rather than the full mask optimization process. Specifically, we will use the Sum of Coherent Systems (SOCS) model, which computes the final light intensity by combining the contributions of multiple optical kernels. The simulation first computes the FFT of the input mask, and then for each optical kernel, multiplies the mask's frequency representation by the kernel, performs an inverse FFT, and computes the resulting light intensity. The intensity contributions from all kernels are then added together, and a resist model is applied to obtain the predicted wafer image.

One issue with this approach is that the optical kernels only use a relatively small portion of the frequencies in the original mask, but a straightforward implementation still performs full-resolution inverse FFTs for every kernel. For example, OpenILT includes an implementation that uses 35×35 optical kernels with 2048×2048 input masks. We plan to implement an existing optimization that takes advantage of this by performing the intermediate calculations at a lower resolution, then reconstructing the final image at its original resolution. This approach has

previously been implemented in K-Litho using C++ and FFTW, and we will investigate how to implement it efficiently on GPUs.

The Challenge

The main challenge of this project is determining how to efficiently execute the lithography simulation on the GPU, especially after reducing the sizes of the intermediate FFTs.

The computations for each optical kernel are independent until their intensity contributions are added together. One approach is to process each kernel separately, while another is to use batched cuFFT to process multiple kernels together. We think batching would reduce the overhead associated with performing many separate FFT operations (it might also improve GPU utilization when the intermediate FFTs are small). It would require additional memory though and might not significantly improve performance if the GPU is already being used efficiently and this is a tradeoff we seek to measure.

After processing the optical kernels, we need to add their intensity contributions at every pixel. Our initial approach will assign one CUDA thread to each pixel, with each thread iterating through the optical modes and computing their sum. We will also experiment with assigning multiple threads to each pixel, where each thread computes a partial sum and the partial sums are then combined. The second approach introduces more parallelism, but it also requires communication between threads, which might make it slower when the number of optical modes is relatively small.

We also want to experiment with different ways of storing the intermediate intensity images in GPU memory. For example, we could store whole images for each optical mode contiguously, or we could store the contributions from different modes for the same pixel together. These layouts would have different memory access characteristics depending on how threads are assigned to pixels and optical modes. We want to compare them to determine which layout provides better memory coalescing and overall performance.

Another optimization we plan to explore is combining operations that would otherwise require separate CUDA kernel launches. For example, our initial implementation could use one kernel to copy the required frequency values into a smaller array and another kernel to multiply them by their optical coefficients. We could instead combine these operations into a single kernel to reduce intermediate memory accesses and kernel launch overhead. We will investigate whether this improves performance, since combining more operations into one kernel could also increase register usage and reduce GPU occupancy.

We expect the largest speedup to come from reducing the size of the intermediate FFTs. However, this optimization could also change the bottlenecks of the simulation, since operations that previously accounted for a small portion of the total runtime might become more significant. We will use profiling to identify these bottlenecks and decide which CUDA optimizations are most useful.

Resources

We have identified several existing codebases and libraries that we will use to benchmark our approach and aid in development. OpenILT is an open-source lithography codebase implemented in PyTorch that contains a lithography simulator, optical kernels and sample inputs which we can use to test our approach. K-Litho is another lithography simulator written in C++ that implements a reduced-FFT algorithm that we can use to ensure our initial algorithmic optimization to the forward pass is correct. We will also use the NVIDIA cuFFT library for computing FFTs on GPUs instead of rebuilding a complete FFT helper. The NVIDIA cuLitho library also implements a lithography simulator in C++ and CUDA but their code is not open-source, so we can also benchmark our final results against theirs.

Goals and Deliverables

Plan to Achieve

1. We will implement a baseline GPU lithography simulator using C++ and CUDA, using the cuFFT library FFT calculations in the forward pass. We will compare its output against OpenILT to test its correctness.
2. We will implement the reduced-resolution simulation algorithm based on the K-Litho codebase, which uses smaller intermediate FFTs and reconstructs the full-resolution intensity image. We will compare its runtime and numerical accuracy against our original GPU implementation.
3. We will implement and compare sequential and batched cuFFT execution for the optical modes, and investigate how performance changes with the number of optical kernels used.
4. We will implement at least two CUDA approaches for accumulating the intensity contributions from the optical modes. We will compare their performance and investigate the overhead associated with having multiple threads cooperate on the same output pixel.
5. We will experiment with different GPU memory layouts and kernel fusion strategies. We will use profiling results to determine which optimizations are most useful rather than assuming that each optimization will improve runtime.
6. We will evaluate the different implementations on multiple input sizes and optical kernel configurations. We will report total simulation runtime, time spent in individual operations, total speedup, and numerical differences compared with the reference implementation.

Hope to Achieve

If we finish our main goals ahead of schedule, we would like to move on to processing multiple masks concurrently using CUDA streams or batched execution. This improvement would be useful for ILT applications, since it would allow us to efficiently simulate multiple input masks at once. We could also integrate our simulator into a full optical proximity correction loop, where

we iteratively modify an input mask based on the error observed in the pattern it projects onto the wafer.

Platform Choice

For this project we will write our code in C++ and CUDA, and we will run our implementation on NVIDIA GPUs on the GHC machines. Using C++ and CUDA will allow us to explicitly control how we assign each thread to a given pixel in the input mask, and we can also easily define different kernels for each operation in the forward pass of the simulator. We can also easily experiment with fusing kernels that perform operations that happen in succession. We are choosing to use GPUs for this project because the lithography simulator must be parallelized over a large number of pixels in the input mask, and we can use cuFFT on GPUs to efficiently calculate FFTs.

Schedule

Dates	Planned Work
Nov 10–15	We plan to familiarize ourselves with the OpenILT implementation, specifically its reference outputs and optical kernels, and begin implementing our baseline C++/CUDA simulator.
Nov 16–22	We plan to complete the baseline simulator and implement the reduced-resolution algorithm for the forward pass of the simulator. We will validate its correctness and collect runtime measurements to present for the November 23 milestone.
Nov 23–29	We plan to implement batched cuFFT and compare our different approaches for parallelizing intensity accumulation. We will also start experimenting with different memory layouts of the intermediate results.
Nov 30–Dec 6	We plan to experiment with kernel fusion and other optimizations we foresee as being useful based on results from profiling. If time permits, we might also begin a stretch goal involving executing multiple input masks in parallel.
Dec 7–10	We plan to run our final benchmarks, compare our implementations quantitatively, analyze remaining performance bottlenecks, and finish writing our final report.
Dec 11	Presentation at Poster Session

References

1. S. Zheng, B. Yu, and M. Wong, *OpenILT: An Open Source Inverse Lithography Technique Framework*, IEEE ASICON, 2023. https://github.com/OpenOPC/OpenILT

2. M. Kuramochi et al., *Development of a Lithography Simulation Tool Set in Various Optical Conditions for Source Mask Optimization*, IEEE Access, 2024. https://doi.org/10.1109/ACCESS.2024.3390936
3. *K-Litho: Lithography Simulation Tool Set.* https://github.com/LithoSimulator/K-Litho
4. NVIDIA, *cuFFT Library Documentation.* https://docs.nvidia.com/cuda/cufft/
5. *ICCAD 2013 CAD Contest: Mask Optimization Benchmark Suite.*